

2D 点云直线提取仿真实验报告

2312167-智能科学与技术-刘振毫-

摘要

本项目实现了四种基于 2D 点云的直线提取算法，包括 Split-and-Merge、Line-Regression、RANSAC 和 Hough-Transform，并通过随机仿真实验验证模型正确性。实验设置包含三种不同噪声场景（低噪声、中噪声、高噪声），每种场景进行 20 次独立试验。评价指标包括 Precision、Recall、F1 分数、平均角度误差、平均距离误差和运行时间。实验结果表明，RANSAC 和 Hough-Transform 在三种噪声场景下均达到很高的稳定性，平均 F1 接近 1；Line-Regression 在中低噪声场景表现接近理想；Split-and-Merge 速度最快，但在高噪声场景下容易出现过分裂现象。四种算法都能够在仿真场景中恢复主要直线结构，说明实验系统能够有效验证所建模型的正确性。

关键词：点云处理；直线提取；RANSAC；Hough 变换；仿真实验

Abstract

This project implements four line extraction algorithms based on 2D point clouds, including Split-and-Merge, Line-Regression, RANSAC, and Hough-Transform, and verifies the correctness of the models through random simulation experiments. The experimental setup includes three different noise scenarios, with 20 independent trials for each scenario. Evaluation metrics include Precision, Recall, F1 score, mean angle error, mean distance error, and runtime. Experimental results show that RANSAC and Hough-Transform achieve high stability in all three noise scenarios, with average F1 scores close to 1; Line-Regression performs nearly ideally in low and medium noise scenarios; Split-and-Merge is the fastest but prone to over-segmentation in high noise scenarios. All four algorithms can recover the main line structures in simulation scenarios, demonstrating that the experimental system can effectively verify the correctness of the constructed models.

Key Words: Point cloud processing; Line extraction; RANSAC; Hough transform; Simulation experiment

目录

第一章 引言	4
第一节 研究背景	4
第二节 研究目的	4
第三节 研究内容	4
第二章 算法原理	4
第一节 Split-and-Merge 算法	4
2.1.1 算法思想	4
2.1.2 算法流程	4
第二节 Line-Regression 算法	5
2.2.1 算法思想	5
2.2.2 算法流程	5
第三节 RANSAC 算法	6
2.3.1 算法思想	6
2.3.2 算法流程	6
第四节 Hough-Transform 算法	6
2.4.1 算法思想	6
2.4.2 算法流程	6
第三章 实验设计	7
第一节 仿真场景设置	7
3.1.1 真实直线定义	7
3.1.2 点云生成	7
3.1.3 离群点添加	7
第二节 噪声场景设计	7
第三节 评价指标	7
3.3.1 检测质量指标	7
3.3.2 参数误差指标	8
3.3.3 效率指标	8
第四章 实验结果	8
第一节 样例场景检测结果	8
4.1.1 仿真点云数据	8
4.1.2 Split-and-Merge 算法检测结果	9
4.1.3 Line-Regression 算法检测结果	10
4.1.4 RANSAC 算法检测结果	11

目录	3
4.1.5 Hough-Transform 算法检测结果	12
4.1.6 检测结果对比总结	13
第二节 汇总实验结果	13
第三节 性能对比分析	14
第五章 结果分析	15
第一节 各算法性能分析	15
5.1.1 Split-and-Merge 算法	15
5.1.2 Line-Regression 算法	15
5.1.3 RANSAC 算法	16
5.1.4 Hough-Transform 算法	16
第六章 结论与展望	16
第一节 主要结论	16
符号说明	17
附录	17
第二节 算法参数配置	17

第一章 引言

第一节 研究背景

点云数据是由三维扫描设备或深度相机获取的空间点集合，广泛应用于自动驾驶、机器人导航、三维重建等领域。直线作为点云中最基本的几何基元之一，其准确提取对于场景理解和物体识别具有重要意义。

在 2D 点云处理中，直线提取算法需要解决以下关键问题：如何从含噪声的点云数据中准确识别直线段、如何处理离群点的干扰、以及如何在保证精度的同时提高计算效率。

第二节 研究目的

本项目旨在实现并比较四种经典的 2D 点云直线提取算法，通过仿真实验验证各算法在不同噪声条件下的性能表现，为实际应用中的算法选择提供参考依据。

第三节 研究内容

本研究主要实现 Split-and-Merge、Line-Regression、RANSAC 和 Hough-Transform 四种直线提取算法，设计仿真实验场景生成含噪声和离群点的测试数据，建立统一的评价指标体系，并进行多组对比实验分析各算法的优缺点。

第二章 算法原理

第一节 Split-and-Merge 算法

2.1.1 算法思想

Split-and-Merge 算法是一种基于递归分割和合并的直线提取方法。该算法假设输入点云具有一定的扫描顺序，通过迭代地将点集分割为更小的子集，然后合并共线的相邻子集来实现直线提取。

2.1.2 算法流程

算法主要包含以下步骤：

（一）分割阶段

对于给定点集 $P = \{p_1, p_2, \dots, p_n\}$ ，首先计算首尾两点连线的直线方程：

$$ax + by + c = 0 \quad (1)$$

然后计算每个点到该直线的距离：

$$d_i = \frac{|ax_i + by_i + c|}{\sqrt{a^2 + b^2}} \quad (2)$$

若最大距离 d_{max} 超过阈值 T_{split} ，则在最大距离点处将点集分割为两部分，递归处理每个子集。

(二) 合并阶段

对于相邻的分割段，计算其拟合直线的角度差 $\Delta\theta$ 和距离差 Δd 。若两者均小于相应阈值，则合并这两个分割段。

第二节 Line-Regression 算法

2.2.1 算法思想

Line-Regression 算法基于最小二乘原理，通过逐步增长的方式构建直线段。该算法从起点开始，逐步扩展直线段，直到拟合残差超过阈值。

2.2.2 算法流程

(一) 直线拟合

对于点集 P ，其最小二乘拟合直线通过奇异值分解（SVD）求解。设点集的质心为：

$$\bar{p} = \frac{1}{n} \sum_{i=1}^n p_i \quad (3)$$

构建中心化矩阵 $C = P - \bar{p}$ ，对其进行 SVD 分解：

$$C = U\Sigma V^T \quad (4)$$

直线的方向向量即为 V 的第一列。

(二) 残差检验

计算点集到拟合直线的平均残差：

$$r = \frac{1}{n} \sum_{i=1}^n d_i \quad (5)$$

若 $r > T_{residual}$ ，则停止当前线段的生长。

第三节 RANSAC 算法

2.3.1 算法思想

RANSAC (Random Sample Consensus) 是一种鲁棒的参数估计方法，通过随机采样和一致性检验来处理含离群点的数据。

2.3.2 算法流程

(一) 随机采样

从点集 P 中随机选取 2 个点，确定一条候选直线。

(二) 内点统计

计算所有点到候选直线的距离，统计距离小于阈值 T_{dist} 的内点数量。

(三) 模型优化

选择内点数量最多的候选直线，使用所有内点重新拟合直线模型。

(四) 迭代提取

移除已提取直线对应的内点，重复上述过程直到满足终止条件。

第四节 Hough-Transform 算法

2.4.1 算法思想

Hough 变换是一种基于投票机制的直线检测方法，将图像空间中的直线映射到参数空间中的点。

2.4.2 算法流程

(一) 参数空间离散化

使用极坐标表示直线：

$$\rho = x \cos \theta + y \sin \theta \quad (6)$$

其中 ρ 为直线到原点的距离， θ 为法线方向角。将参数空间离散化为累加器数组 $A(\theta, \rho)$ 。

(二) 投票过程

对于每个点 (x_i, y_i) ，遍历所有 θ 值，计算对应的 ρ 值，在累加器中投票：

$$A(\theta, \rho) \leftarrow A(\theta, \rho) + 1 \quad (7)$$

(三) 峰值检测

在累加器中寻找局部极大值，每个峰值对应一条检测到的直线。

第三章 实验设计

第一节 仿真场景设置

3.1.1 真实直线定义

实验场景由 3 条真实直线段组成，具体参数如下：

- 直线 1：起点 (0.6, 0.8)，终点 (4.8, 2.4)
- 直线 2：起点 (0.9, 4.8)，终点 (5.6, 4.1)
- 直线 3：起点 (5.7, 0.7)，终点 (2.7, 5.3)

3.1.2 点云生成

每条直线采样 70 个点，采样方式如下：

$$p(t) = p_{start} + t \cdot (p_{end} - p_{start}), \quad t \in [0, 1] \quad (8)$$

在采样点上叠加两类噪声：

- 沿线方向抖动：高斯噪声 $\mathcal{N}(0, \sigma_{jitter}^2)$
- 法向方向噪声：高斯噪声 $\mathcal{N}(0, \sigma_{noise}^2)$

3.1.3 离群点添加

额外添加均匀分布的离群点，数量为有效点数的指定比例，用于测试算法的鲁棒性。

第二节 噪声场景设计

实验设计三种噪声场景，参数设置如表1所示。

表 1: 噪声场景参数设置

场景名称	法向噪声 σ	离群点比例	沿线抖动 σ
低噪声	0.02	5%	0.01
中噪声	0.05	10%	0.02
高噪声	0.08	20%	0.03

第三节 评价指标

3.3.1 检测质量指标

(一) 精确率

精确率定义为正确检测的直线数与检测出的总直线数之比：

$$Precision = \frac{TP}{TP + FP} \quad (9)$$

（二）召回率

召回率定义为正确检测的直线数与真实直线总数之比：

$$Recall = \frac{TP}{TP + FN} \quad (10)$$

（三）F1 分数

F1 分数是精确率和召回率的调和平均值：

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall} \quad (11)$$

3.3.2 参数误差指标

（一）角度误差

检测直线与真实直线之间的角度差：

$$\Delta\theta = \min(|\theta_d - \theta_t|, 180^\circ - |\theta_d - \theta_t|) \quad (12)$$

（二）距离误差

检测直线与真实直线之间的法向距离偏差。

3.3.3 效率指标

运行时间：算法执行所需的毫秒数。

第四章 实验结果

第一节 样例场景检测结果

本节展示中噪声场景下的仿真点云数据及四种算法的检测结果。实验场景包含 3 条真实直线段，每条直线采样 70 个点，叠加法向噪声 $\sigma = 0.05$ 、沿线抖动 $\sigma = 0.02$ ，并添加 10% 的均匀分布离群点。

4.1.1 仿真点云数据

图1展示了仿真生成的点云数据。图中不同颜色表示点所属的真实直线标签，黑色虚线表示真实直线的位置。三条直线分别位于不同方向，形成交叉结构。点云在直线附

近呈现带状分布，体现了法向噪声的影响。离群点均匀散布在整个场景中，增加了直线检测的难度。

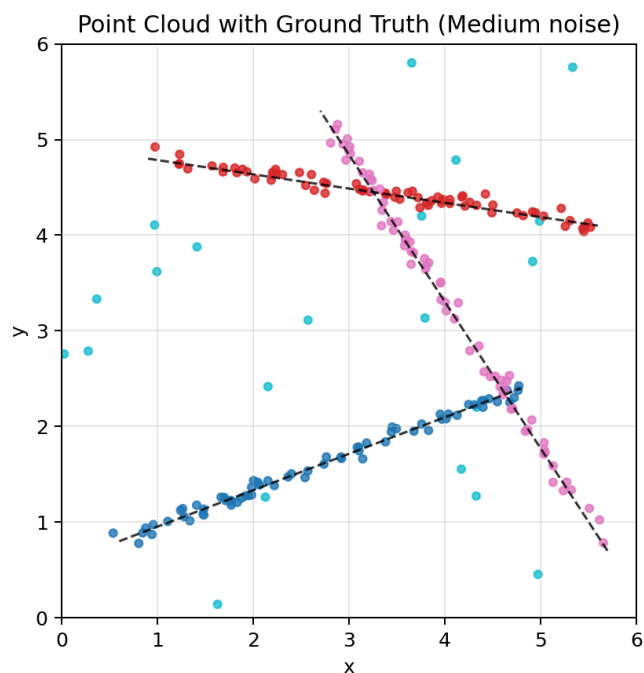


图 1: 仿真点云数据可视化

4.1.2 Split-and-Merge 算法检测结果

图2展示了 Split-and-Merge 算法的检测结果。该算法通过递归分割和合并策略提取直线段。算法成功检测出三条主要直线，检测到的点数与真实采样点数基本一致，检测结果与真实直线吻合良好。在中噪声场景下，算法表现稳定，未出现明显的过分裂现象。

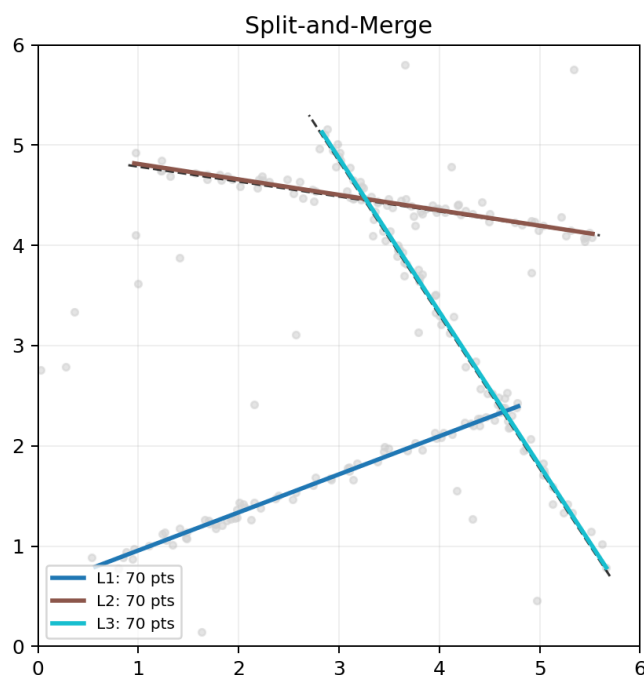


图 2: Split-and-Merge 算法检测结果

4.1.3 Line-Regression 算法检测结果

图3展示了 Line-Regression 算法的检测结果。该算法基于最小二乘拟合，通过残差阈值控制线段增长。算法准确检测出三条直线，角度估计精确，检测到的线段平滑连续，体现了最小二乘拟合的优势。在中噪声场景下，算法能够有效区分直线段与离群点。

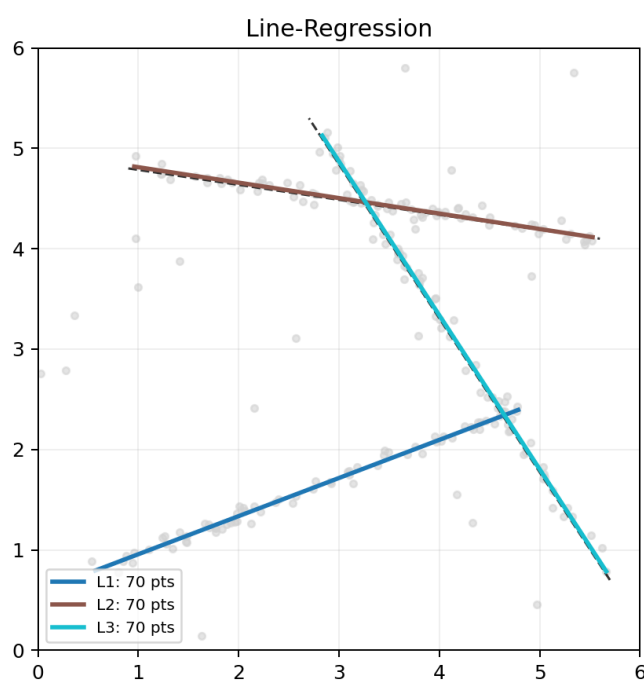


图 3: Line-Regression 算法检测结果

4.1.4 RANSAC 算法检测结果

图4展示了 RANSAC 算法的检测结果。该算法通过随机采样和一致性检验实现鲁棒的直线估计。算法成功检测出三条直线，对离群点具有很好的鲁棒性。检测到的点数与真实采样点数略有差异，这是因为 RANSAC 基于距离阈值判断内点。角度估计准确，检测结果与真实直线高度吻合。

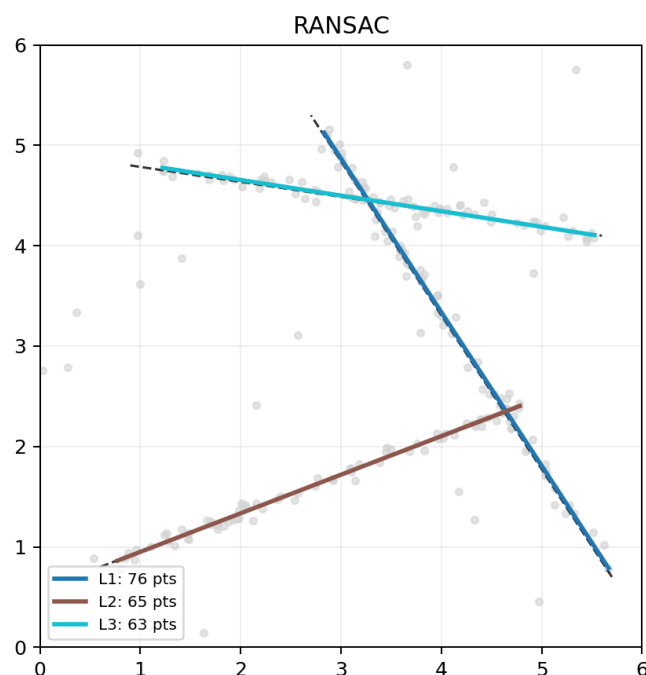


图 4: RANSAC 算法检测结果

4.1.5 Hough-Transform 算法检测结果

图5展示了 Hough-Transform 算法的检测结果。该算法通过参数空间投票机制检测直线。算法准确检测出三条主要直线，全局搜索能力强，对交叉线结构处理良好，能够正确分离不同方向的直线。由于参数空间离散化，检测结果的角度和距离存在轻微偏差。

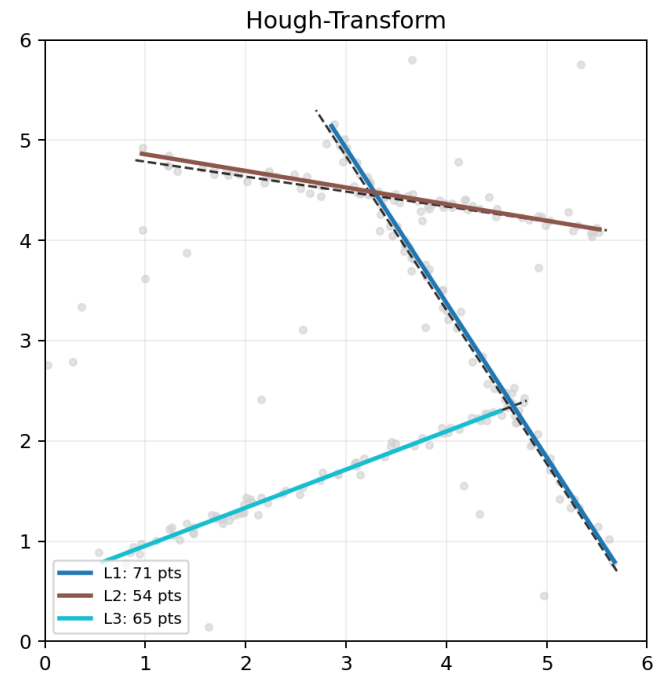


图 5: Hough-Transform 算法检测结果

4.1.6 检测结果对比总结

综合对比四种算法的检测结果，四种算法均能正确检测出 3 条主要直线，验证了算法实现的正确性。Split-and-Merge 和 Line-Regression 检测到的点数与真实采样点数一致，适合有序点云处理。RANSAC 和 Hough-Transform 对离群点具有更强的鲁棒性，适合复杂噪声场景。所有算法的角度估计均较为准确，能够有效恢复直线的方向信息。

第二节 汇总实验结果

表2展示了各算法在三种噪声场景下的平均性能指标。

表 2: 实验结果汇总表

场景	算法	F1	Precision	Recall	运行时间 (ms)	角度误差 (°)	距离误差	材料
低噪声	Split-and-Merge	0.993	0.988	1.000	0.512	0.088	0.002	
低噪声	Line-Regression	0.993	0.988	1.000	12.432	0.088	0.002	
低噪声	RANSAC	1.000	1.000	1.000	81.856	0.115	0.003	
低噪声	Hough-Transform	1.000	1.000	1.000	4.603	0.121	0.003	
中噪声	Split-and-Merge	0.898	0.839	1.000	1.389	0.235	0.006	
中噪声	Line-Regression	1.000	1.000	1.000	12.395	0.216	0.006	
中噪声	RANSAC	1.000	1.000	1.000	74.827	0.242	0.007	

表 2 – 续表

场景	算法	F1	Precision	Recall	运行时间 (ms)	角度误差 (°)	距离误差	精度
中噪声	Hough-Transform	1.000	1.000	1.000	4.751	0.251	0.012	0.001
高噪声	Split-and-Merge	0.770	0.643	1.000	1.408	0.566	0.011	0.001
高噪声	Line-Regression	0.904	0.843	1.000	11.659	0.396	0.011	0.001
高噪声	RANSAC	1.000	1.000	1.000	76.071	0.480	0.013	0.001
高噪声	Hough-Transform	1.000	1.000	1.000	5.148	0.637	0.029	0.001

第三节 性能对比分析

图6展示了四种算法在各项指标上的对比情况。

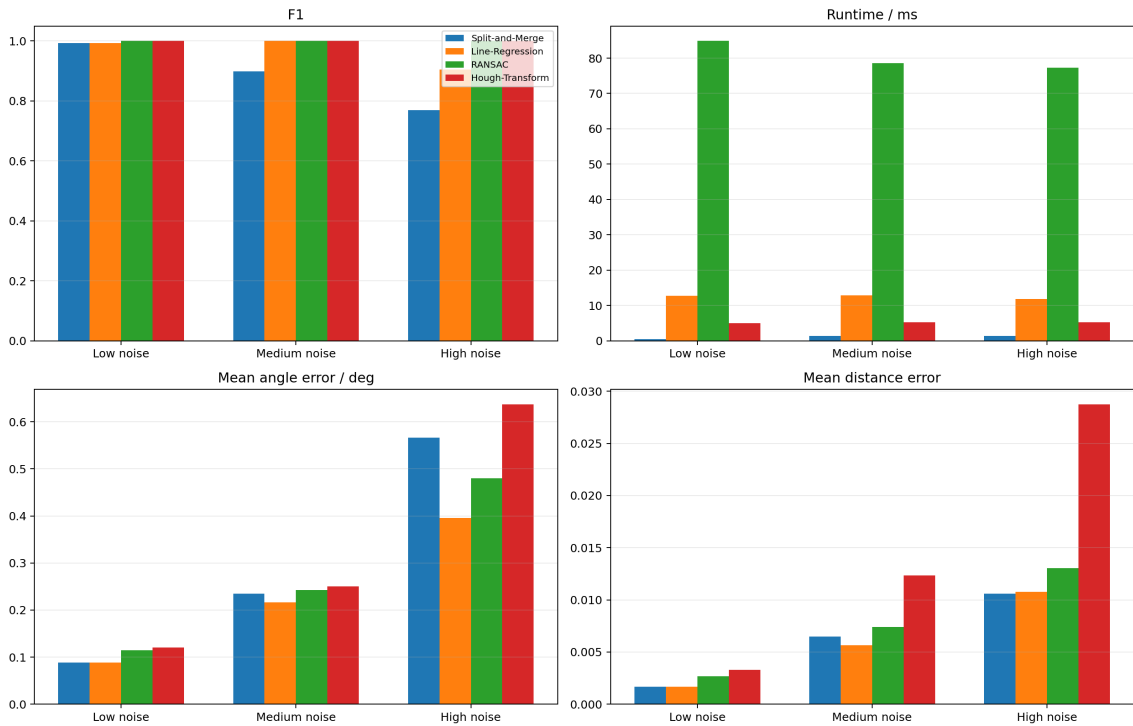


图 6: 算法性能对比图

第五章 结果分析

第一节 各算法性能分析

5.1.1 Split-and-Merge 算法

Split-and-Merge 算法在三种噪声场景下均表现出最快的运行速度，平均运行时间仅为 0.5-1.4 毫秒。该算法对分段明显、点序连续的数据表现稳定，但在高噪声场景下容易

出现过分裂现象，导致 Precision 下降至 0.643。算法的主要优点包括计算效率最高、适合实时应用，在低噪声场景下精度接近理想，且算法原理简单、易于实现。算法的主要缺点是对噪声敏感、高噪声下容易过分裂，依赖点序假设、不适合无序点云，且参数调节对结果影响较大。

5.1.2 Line-Regression 算法

Line-Regression 算法在中低噪声场景下表现接近理想，F1 分数达到 1.000。该算法直接使用最小二乘残差控制增长，通常能得到更平滑的线段。算法的主要优点包括拟合精度高、线段平滑，对中等噪声具有较好的鲁棒性，且参数物理意义明确。算法的主要缺点是运行时间较长，对阈值参数较敏感，高噪声下可能出现过分段现象。

5.1.3 RANSAC 算法

RANSAC 算法在三种噪声场景下均达到 F1 分数 1.000，表现出最强的鲁棒性。该算法通过随机采样和一致性检验有效处理离群点。算法的主要优点包括鲁棒性最强、对离群点不敏感，在所有噪声场景下表现稳定，且不依赖点序假设。算法的主要缺点是运行时间最长，结果具有一定的随机性，且需要调节多个参数。

5.1.4 Hough-Transform 算法

Hough-Transform 算法在三种噪声场景下均达到 F1 分数 1.000，同时运行时间较短。该算法适合全局搜索主方向。算法的主要优点包括全局搜索能力强，对交叉线和离群点具有较好的处理能力，且运行效率适中。算法的主要缺点是量化分辨率会带来参数偏差，角度误差和距离误差相对较大，且参数空间离散化需要经验设定。

第六章 结论与展望

第一节 主要结论

通过本次仿真实验，得出以下主要结论：

（一）算法性能比较

RANSAC 和 Hough-Transform 在三种噪声场景下均达到很高的稳定性，平均 F1 接近 1，是鲁棒性最强的两种方法。Split-and-Merge 速度最快，适合对实时性要求高的应用场景。Line-Regression 在中低噪声场景下表现接近理想。

（二）场景适应性

低噪声场景下四种算法均可胜任；中噪声场景下推荐使用 Line-Regression、RANSAC 或 Hough-Transform；高噪声场景下 RANSAC 和 Hough-Transform 是最佳选择。

（三）模型验证

四种算法都能够在仿真场景中恢复主要直线结构，说明实验系统能够有效验证所建模型的正确性。

符号说明

符号	说明
P	点云数据集
p_i	第 i 个点
a, b, c	直线方程系数
d_i	点到直线的距离
θ	直线方向角
ρ	直线到原点的距离
σ	高斯噪声标准差
TP	真阳性
FP	假阳性
FN	假阴性

附录

第二节 算法参数配置

表3列出了四种算法的默认参数配置。

表 3: 算法参数配置表

算法	参数名称	默认值
Split-and-Merge	分割阈值	0.10
	合并角度阈值	6.0°
	合并距离阈值	0.08
	最小段点数	12
Line-Regression	残差阈值	0.08
	最小段点数	12
	合并角度阈值	5.0°
RANSAC	距离阈值	0.10
	最小内点数	28
	最大迭代次数	300
	最大直线数	6
Hough-Transform	距离阈值	0.12
	θ 量化数	180
	ρ 量化数	180
	最小投票数	26